AgriQuest - Gamified Sustainable Farming Platform

Complete Build Guide from Start to End

Table of Contents

1. Project Overview
2. Technology Stack
3. System Architecture
4. Prerequisites
5. Installation Steps
6. Database Setup
7. Environment Configuration
8. AI Integration Setup
9. Running the Application
10. Testing
11. Deployment Options
12. API Documentation
13. Features Walkthrough
14. Troubleshooting

---

1. Project Overview

AgriQuest is a comprehensive platform that empowers farmers with sustainable farming practices through gamification, AI-powered crop analysis, and community engagement.

Key Features:

- **Gamified Farming**: Earn points, badges, and rewards for completing sustainable farming missions
- **AI Crop Analysis**: Upload photos of your crops for health analysis and recommendations
- **Government Auction Integration**: Apply for government auctions with verified crop reports
- **Multilingual Voice/Chat Bot**: Get farming advice in your local language
- **Community Platform**: Share experiences and learn from other farmers
- **Leaderboards**: Compete with other farmers in sustainability challenges
- **Offline Support**: Works even without internet connection

---

2. Technology Stack

Frontend:

- **Framework**: Next.js 15 with App Router
- **Styling**: Tailwind CSS
- **Language**: TypeScript
- **UI Components**: React with Lucide React icons
- **State Management**: React Hooks
- **Internationalization**: i18next

Backend:

- **Runtime**: Node.js with Express
- **Language**: TypeScript
- **Database**: PostgreSQL with Prisma ORM
- **Authentication**: Phone-based OTP system
- **Real-time Features**: Socket.IO

AI & Machine Learning:

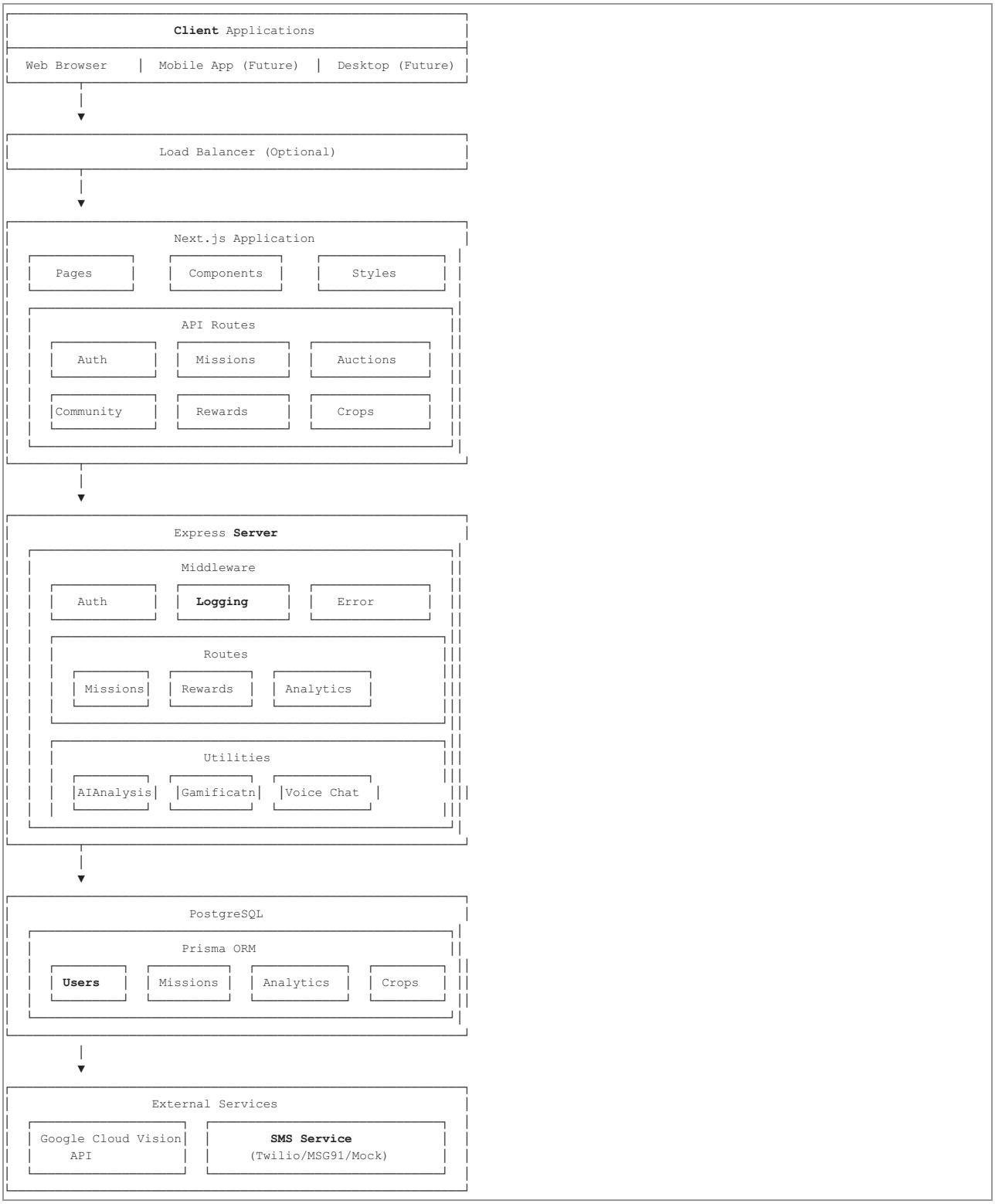
- **TensorFlow.js**: For client-side crop analysis
- **Google Cloud Vision API**: For advanced image analysis
- **MobileNet & COCO-SSD**: Pre-trained models for plant detection

Deployment:

- **Containerization**: Docker
- **Orchestration**: Docker Compose
- **Cloud Platforms**: Render, Vercel, Netlify

---

3. System Architecture



---

4. Prerequisites

Before installing AgriQuest, ensure you have the following software installed:

1. **Node.js** (v18 or higher)
2. **npm** or **yarn**
3. **Docker** (for database - optional for demo)
4. **Git** (for version control)
5. **PostgreSQL** (v12 or higher - if not using Docker)
6. **Google Cloud Account** (for Google Vision API - optional)

---

5. Installation Steps

Step 1: Clone the Repository

```
git clone <repository-url>
cd agriquest-app
```

Step 2: Install Dependencies

```
npm install
```

This will install all required dependencies including:

- Next.js for the frontend
- Express for the backend
- Prisma for database ORM
- TensorFlow.js for AI analysis
- Tailwind CSS for styling

Step 3: Generate Prisma Client

```
npm run db:generate
```

This generates the Prisma client based on the schema defined in `prisma/schema.prisma`.

---

6. Database Setup

AgriQuest uses PostgreSQL as its primary database. You have two options for setting up the database:

Option 1: Using Docker (Recommended for Development)

1. Start the database container:

```
docker-compose up -d
```

2. Run database migrations:

```
npm run db:migrate
```

Option 2: Using Local PostgreSQL Installation

1. Create a new PostgreSQL database:

```
CREATE DATABASE agriquest_db;
CREATE USER agriquest_user WITH ENCRYPTED PASSWORD 'agriquest_password';
GRANT ALL PRIVILEGES ON DATABASE agriquest_db TO agriquest_user;
```

2. Update your `.env.local` file with the database connection string:

```
DATABASE_URL=postgresql://agriquest_user:agriquest_password@localhost:5432/agriquest_db
```

3. Run database migrations:

```
npm run db:migrate
```

Seeding the Database

To populate the database with initial data:

```
npm run db:seed
```

---

7. Environment Configuration

Create a `.env.local` file in the root directory with the following configuration:

```
# Database Configuration
DATABASE_URL=postgresql://username:password@localhost:5432/agriquest_db

# JWT Secret for authentication (min 32 characters)
JWT_SECRET=your-super-secret-jwt-key-change-this-in-production-min-32-characters

# Server Configuration
PORT=3000
NEXT_PUBLIC_API_URL=http://localhost:3000

# AI Configuration
NEXT_PUBLIC_TFJS_MODEL_URL=/models/model.json
GOOGLE_APPLICATION_CREDENTIALS=./config/service-account-key.json
GOOGLE_VISION_ENABLED=false
AI_IMPLEMENTATION=tfjs

# Development only - set to true in production
NEXT_PUBLIC_ENABLE_MOCK_DATA=false

# I18n Configuration
NEXT_PUBLIC_DEFAULT_LANGUAGE=en
```

For a quick demo setup, you can copy the `.env.example` file:

```
cp .env.example .env.local
```

8. AI Integration Setup

AgriQuest supports two AI implementations for crop analysis:

Option 1: TensorFlow.js (Default - Client-side)

No additional setup required. TensorFlow.js models are loaded directly in the browser.

Option 2: Google Cloud Vision API (Server-side)

1. Create a Google Cloud Project
2. Enable the Vision API
3. Create a service account and download the JSON key file
4. Place the key file in `config/service-account-key.json`
5. Update your `.env.local`:

```
GOOGLE_VISION_ENABLED=true
AI_IMPLEMENTATION=google-vision
GOOGLE_APPLICATION_CREDENTIALS=./config/service-account-key.json
```

9. Running the Application

Development Mode

To run the application in development mode with both frontend and backend:

```
npm run dev
```

This will start:

- Frontend server on http://localhost:3000
- Backend API routes integrated within the Next.js app

Production Mode

To build and run the application in production mode:

```
npm run build
npm start
```

Using the Startup Scripts

For Windows users, you can use the PowerShell scripts:

```
./start-full-app.ps1
```

10. Testing

Running Unit Tests

```
npm test
```

Running Specific Test Suites

```
# Test AI analysis functionality
npm run test -- __tests__/ai-analysis.test.ts

# Test missions functionality
npm run test -- __tests__/missions.test.ts
```

API Testing

To test API endpoints:

```
npm run test-api
```

11. Deployment Options

Option 1: Render (Recommended)

1. Fork the repository to your GitHub account
2. Create a new Web Service on Render
3. Connect your GitHub repository
4. Set the following environment variables in Render:
    - `JWT_SECRET` (generate a secure secret)
    - `GOOGLE_VISION_ENABLED` (set to "true" if using Google Vision)
5. Deploy!

Option 2: Vercel

1. Push your code to a GitHub repository
2. Create a new project on Vercel

3. Connect your GitHub repository
4. Configure environment variables in the Vercel dashboard
5. Deploy!

Option 3: Docker Deployment

1. Build the Docker image:

```
docker build -t agriquest .
```

2. Run the container:

```
docker run -p 3000:3000 agriquest
```

Option 4: Traditional Server Deployment

1. Build the application:

```
npm run build
```

2. Start the application:

```
npm start
```

---

12. API Documentation

Authentication Endpoints

- `POST /api/auth/send-otp` - Send OTP to phone number
- `POST /api/auth/verify-otp` - Verify OTP and login
- `POST /api/auth/logout` - Logout user

User Management

- `GET /api/users/profile` - Get user profile
- `PUT /api/users/profile` - Update user profile
- `GET /api/users/farmer` - Get farmer details
- `PUT /api/users/farmer` - Update farmer details

Missions System

- `GET /api/missions` - Get available missions
- `GET /api/missions/:id` - Get specific mission details
- `POST /api/missions/:id/assign` - Assign mission to farmer
- `PUT /api/missions/:id/progress` - Update mission progress
- `POST /api/missions/:id/complete` - Complete mission

Rewards System

- `GET /api/rewards` - Get available rewards
- `POST /api/rewards/:id/redeem` - Redeem reward

Leaderboard

- `GET /api/leaderboard` - Get leaderboard entries

Community

- `GET /api/community/posts` - Get community posts
- `POST /api/community/posts` - Create new post
- `POST /api/community/posts/:id/like` - Like a post

Crop Tracking

- `POST /api/crop-tracking` - Create new crop tracking entry
- `GET /api/crop-tracking/:id` - Get crop tracking details
- `POST /api/crop-tracking/:id/photos` - Upload crop photos
- `POST /api/crop-tracking/ai-analysis` - Analyze crop health

AI Analysis

- `POST /api/crop-tracking/ai-analysis` - Analyze crop health from image
- `POST /api/crop-tracking/compare-growth` - Compare growth over time

Auction System

- `GET /api/auctions` - Get available auctions
- `POST /api/auctions/apply` - Apply for auction with crop report
- `GET /api/auctions/:id` - Get auction details

Voice Chat

- `POST /api/voice-chat/transcribe` - Transcribe voice input
- `POST /api/voice-chat/respond` - Get voice response

---

13. Features Walkthrough

Authentication & Onboarding

1. Users can authenticate using phone-based OTP system
2. Comprehensive farmer profile setup with farm details
3. Crop information and farming experience tracking

Dashboard

1. Overview of points, streaks, and sustainability score
2. Quick access to all major features
3. Recent activity feed

Missions System

1. Step-by-step farming missions tailored to crops and region
2. Photo documentation for verification
3. Points and rewards for completion

AI Crop Analysis

1. Upload crop photos for health analysis
2. Receive detailed reports on health score, leaf color, density, etc.
3. Actionable recommendations for improvement

Crop Reports

1. Generate comprehensive crop credit reports
2. Visualize growth timelines and sustainability metrics
3. Share reports with government agencies for auctions

Government Auction Flow

1. Apply for government auctions with verified reports
2. Track application status
3. Receive notifications about auction results

Multilingual Voice/Chat Bot

1. Voice input in multiple languages
2. Text-to-speech responses in the same language
3. Farming advice and support

Leaderboard & Community

1. Regional and global leaderboards
2. Community posts and discussions
3. Knowledge sharing platform

Rewards System

1. Redeem points for seeds, tools, training, and cash rewards
2. Track reward status and delivery

---

14. Troubleshooting

Common Issues

1. **Port Conflicts**: If ports are in use, the application will automatically use different ports. Check the terminal output for the actual URLs.
2. **Database Connection**: For demo purposes, the application uses a mock authentication system that doesn't require a database. If you want to use the full database features, ensure Docker is running and the database is properly configured.
3. **TypeScript Errors**: If you encounter TypeScript errors, try:

```
npm run build
```

4. **AI Model Loading Issues**: If TensorFlow.js models fail to load, ensure you have a stable internet connection. The models are loaded from CDN.
5. **Environment Variables**: Make sure all required environment variables are set in your `.env.local` file.

Demo Mode

The application is fully functional in demo mode without a database connection. All features are accessible through the demo account.

For hackathon presentations and quick testing, you can access the demo account:

1. Visit http://localhost:3000/demo-login for instant access
2. Or use the phone number `9876543210` with OTP `123456` on the regular login page

The demo account includes:

- Pre-filled farmer profile with realistic data
- 1500 points and level 3 status
- 7-day streak and 85 sustainability score
- Completed missions and achievements

Conclusion

AgriQuest is a comprehensive platform that combines modern web technologies with agricultural best practices to create a gamified farming experience. With its modular architecture, it can be easily extended and customized for different regions and farming practices.

The platform's use of AI for crop analysis, community features for knowledge sharing, and gamification elements for engagement makes it a powerful tool for promoting sustainable farming practices among farmers worldwide.